

Thinking in Types

Ein mentales Modell für TypeScript

Claude Jordan

Typen sind Mengen

TypeScript ist eine funktionale
Programmiersprache

TypeScript arbeitet auf Mengen

C prüft den Namen

```
typedef struct {  
    char* name;  
} Dog;  
  
typedef struct {  
    char* name;  
} Cat;  
  
void greet(Dog d) { printf("Hello, %s\n", d.name); }  
  
Dog d = { "Beethoven" };  
greet(d); // ✓  
  
Cat c = { "Jennifer" };  
greet(c); // ✗
```

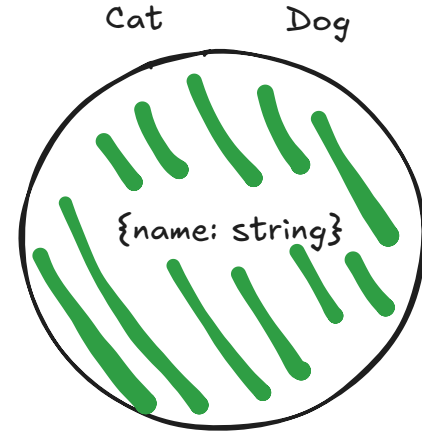
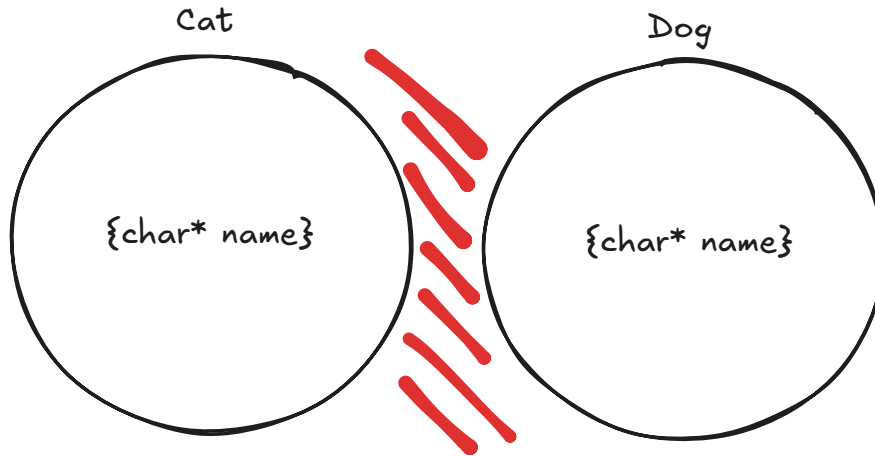
TypeScript prüft die Form

```
type Dog = { name: string }  
type Cat = { name: string }  
  
function greet(d: Dog) { console.log("Hello, " + d.name )  
  
const d: Dog = { name: "Beethoven" }  
greet(d) // ✓  
  
const c: Cat = { name: "Jennifer" }  
greet(c) // ✓
```

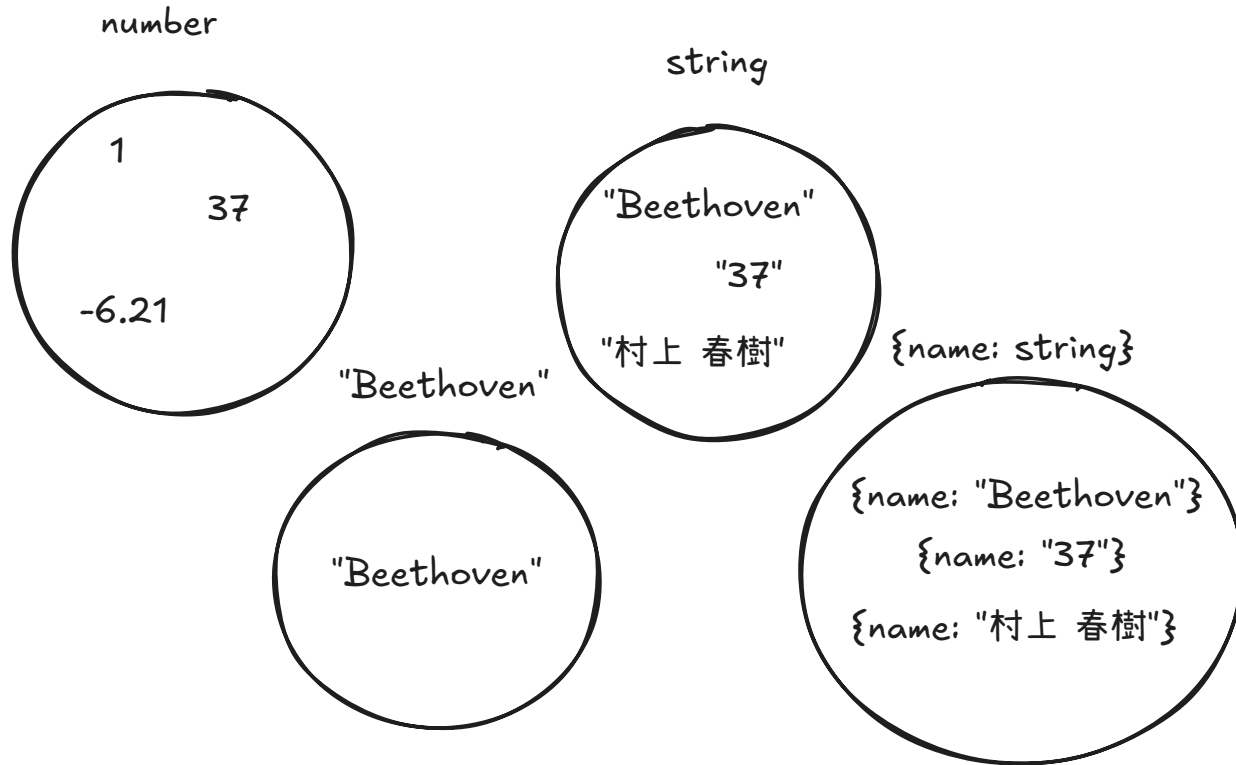
Nominal vs. strukturell

In C: ein Typ wird durch seinen Namen beschrieben

In TypeScript: ein Typ wird durch seine Struktur beschrieben



Jeder Typ ist eine Menge von Werten



Jeder Typ ist eine Menge von Werten

{name: string}

{name: "Beethoven"}

{name: "Jennifer", age: 25}

{name: "Mario",
enemies: [...],
hobbies: {...}
}

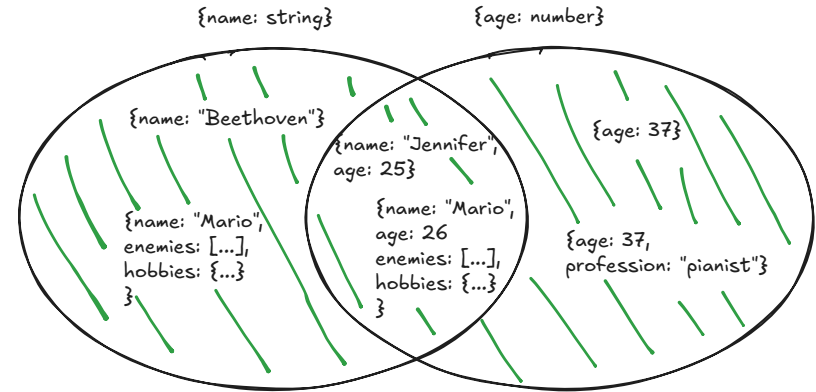
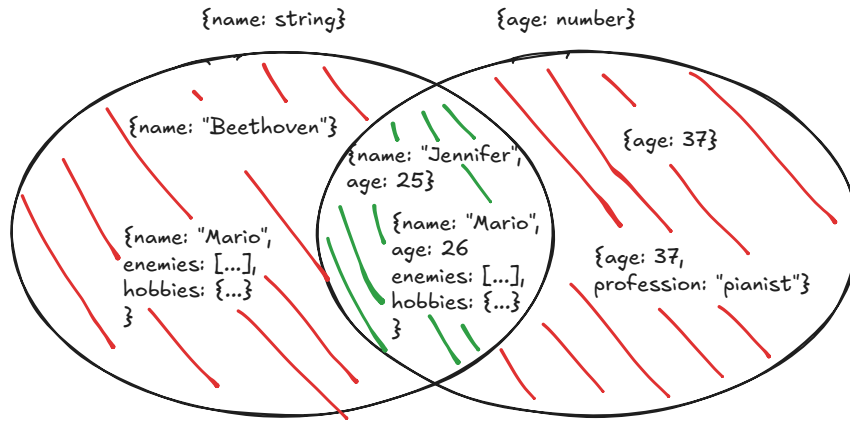
{age: number}

{age: 37}

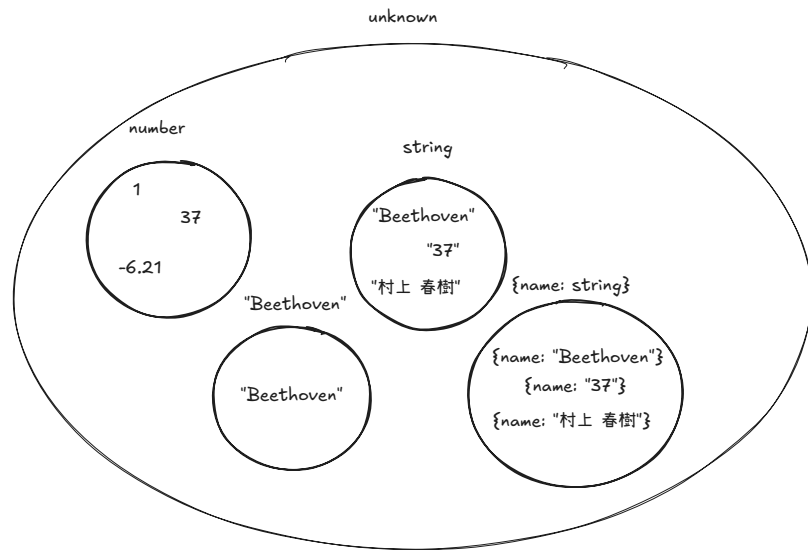
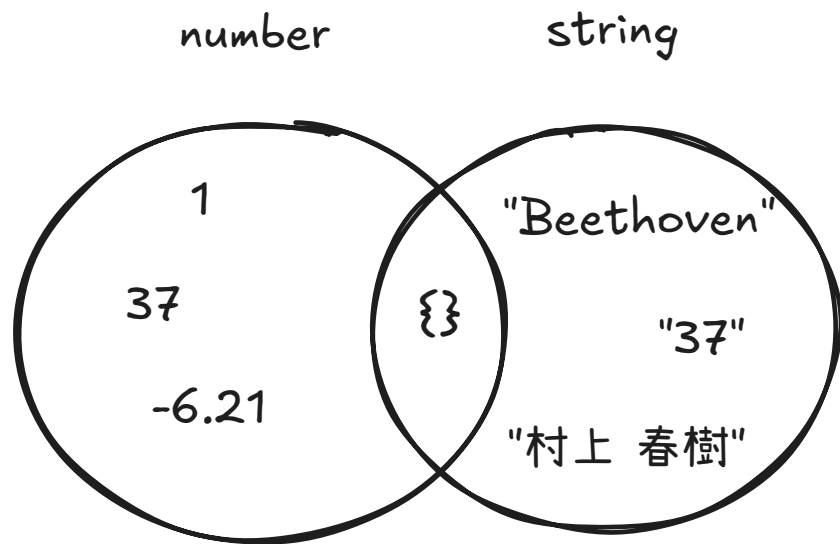
{name: "Jennifer", age: 25}

{age: 37, profession: "pianist"}

Intersection und Union

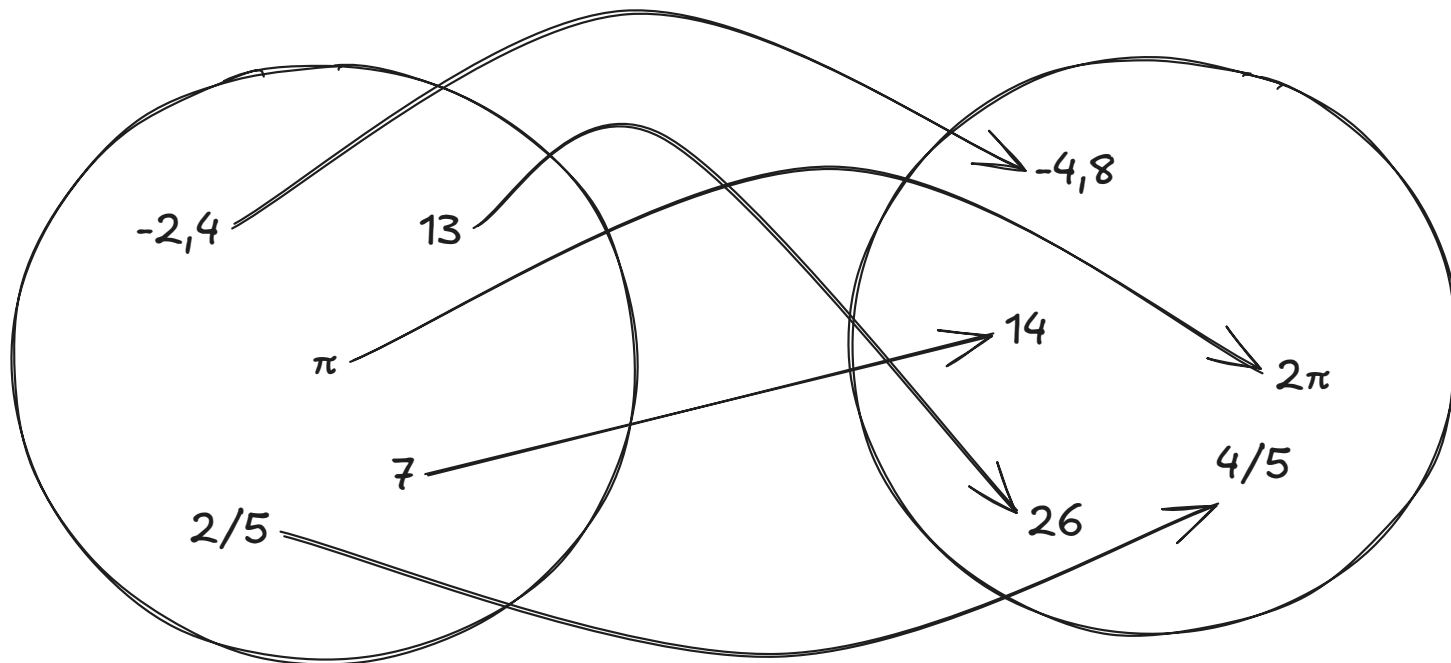


never und unknown



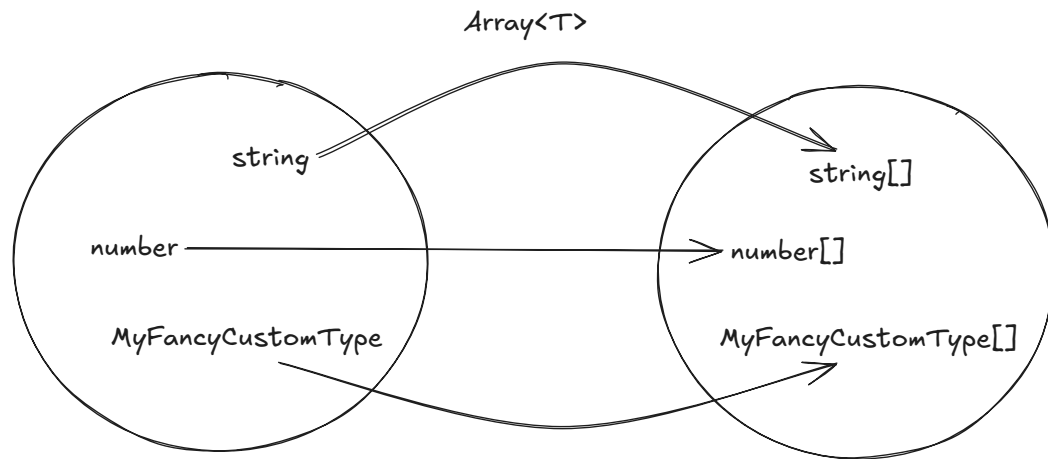
Funktionen sind Abbildungen auf Mengen

$$f(x) = 2x$$



Generics

```
type Array<T> = T[]  
type MyNumberArray = Array<number> // number[]  
  
function toArray( ... numbers ) {  
  return numbers  
}
```



Branching: Conditional Types & infer

```
type ElementType<T> = T extends (infer U)[] ? U : T
```

```
type A = ElementType<number>    // number
type B = ElementType<string[]> // string
type C = ElementType<string[][]> // string[]
```

```
function elementType(value) {
  return Array.isArray(value) ? value[0] : value;
}
```

```
elementType(42);           // 42
elementType(["a", "b"]);   // "a"
elementType(["nested", "array", "outside"]) // ["nested", "array"]
```

- `T extends X` = "ist T eine **Teilmenge** von X?"
- `infer` = frag TypeScript, welcher Typ passt

Verschachtelung auflösen...?

```
type NestedElementType<T> = T extends (infer U)[]  
  ? U extends (infer V)[]  
    ? V  
    : U  
  : T  
  
type T = NestedElementType<string[][]> // string
```

Geht das eleganter...?

Rekursion

```
type DeepElementType<T> =  
  T extends (infer U)[]  
    ? DeepElementType<U>  
    : T;
```

```
type A = DeepElementType<number>    // number  
type B = DeepElementType<string[]>  // string  
type C = DeepElementType<string[][]> // string
```

```
function deepElementType(value) {  
  return Array.isArray(value) ? deepElementType(value[0]) : value;  
}
```

```
DeepElementType(42);           // 42  
DeepElementType(["a", "b"]);   // "a"  
DeepElementType([["nested", "array"], "outside"]) // "nested"
```

Mapped Types

```
type OnlyStringsAndBoolsAllowed = { [key: string]: string | boolean };  
// {a: "hello", myBoolean: true}
```

Mit Teilmengen + Generics:

```
type FamousComposers = "Beethoven" | "Mozart" | "Bach";  
type MappedComposers = { [K in Lowercase<FamousComposers>]: K };  
  
// {beethoven: "beethoven", mozart: "mozart", bach: "bach"}
```

Ziel: kleinste Menge, die Typen
vollständig beschreibt

Discriminated Unions

```
type ApiResponse =  
  | { status: 'loading' }  
  | { status: 'success'; data: User[] }  
  | { status: 'error'; message: string }
```

Jede Branch ist eine disjunkte Teilmenge – unterschieden durch `status`.

Narrowing ist Mengen-Verkleinerung

```
function render(res: ApiResponse) {  
  // res: loading | success | error  - die ganze Menge  
  if (res.status === 'success') {  
    res.data    // ✅ nur im success-Branch sichtbar  
  }  
}
```

Jeder Type Guard verkleinert die Menge: `===` · `typeof` · `instanceof` · `in` · Truthiness

Exhaustiveness mit `never`

```
function render(res: ApiResponse): string {  
  switch (res.status) {  
    case 'loading': return 'Lädt...'  
    case 'success': return `${res.data.length} Einträge`  
    case 'error':    return res.message  
    default:  
      const _exhaustive: never = res // ❌ nicht alle Fälle abgedeckt  
  }  
}
```

Branded Types

```
type Brand<T, B> = T & { readonly _brand: B }  
type Email = Brand<string, 'Email'>  
  
function isEmail(value: string): value is Email {  
  return value.includes('@')  
}  
  
function sendWelcome(to: Email) {  
  ...  
}  
  
const input = 'foo@bar.com'  
sendWelcome(input)      // ❌ string ist keine Email  
if (isEmail(input)) {  
  sendWelcome(input)    // ✅ verengt auf die Email-Teilmenge  
}
```

Template Literal Types

```
type EventName = `on${Capitalize<string>}`  
// 'onClick' | 'onChange' | 'onSubmit' | ...
```

Excess Property Checking

```
type Point = { x: number; y: number }  
  
const obj = { x: 1, y: 2, z: 3 }  
const p: Point = obj //   
  
const q: Point = { x: 1, y: 2, z: 3 } //  excess property 'z'
```

Laut Mengenmodell sind beide gültig. TypeScript ist bei frischen Object Literals bewusst strenger.

Routen-Parameter Parser

```
type Params<T extends string> =  
  T extends `${string}:${infer Param}/${infer Rest}`  
    ? { [K in Param]: string } & Params<`${Rest}`>  
    : T extends `${string}:${infer Param}`  
      ? { [K in Param]: string }  
      : {}  
  
type R = Params<"/users/:id/posts/:postId">  
// { id: string; postId: string }
```

Zusammenfassung

Strukturell statt nominal → Typen sind Mengen von Werten

- Intersection & Union · `never` ($\emptyset$) & `unknown`

TypeScript ist eine funktionale Sprache auf Mengen

- Generics = Funktionen · Conditional Types & `infer` · Rekursion · Mapped Types

In der Praxis

- Discriminated Unions · Narrowing · Exhaustiveness mit `never`
- Branded Types · Template Literal Types · Excess Property Checking
- Routen-Parser – alles zusammen

Das Ziel: kein Regelwerk auswendig lernen – ein Modell haben, aus dem die Regeln folgen.

Danke!

Fragen?